

实验4：陷阱机制（Traps）

• 学号2351232 姓名魏义乾

目录

- 实验4：陷阱机制（Traps）
 - 目录
 - 实验得分
 - 实验概述
 - Task 1: RISC-V Assembly Analysis (Easy)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得
 - Task 2: Backtrace Implementation (Moderate)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得
 - Task 3: Alarm System Implementation (Hard)
 - 实验目的
 - 实验步骤
 - 实验结果
 - 实验中遇到的问题及解决方法
 - 实验心得
-

实验得分

在 traps 分支下执行评分命令：

```
make grade
```

得分结果：

```
qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$ tcpdump -XXnr packets.pcap
Warning: setgid/setuid failed !
reading from file packets.pcap, link-type EN10MB (Ethernet)
21:37:40.284915 IP 10.0.2.15.2003 > 10.0.2.2.26099: UDP, length 5
    0x0000:  5255 0a00 0202 5254 0012 3456 0800 4500  RU....RT..4V..E.
    0x0010:  0021 0000 0000 6411 3ebc 0a00 020f 0a00  .!....d.>.....
    0x0020:  0202 07d3 65f3 000d 0000 7478 6f6e 65    ....e.....txone
21:38:59.385065 IP6 fe80::2 > ff02::1: ICMP6, router advertisement, length 56
    0x0000:  3333 0000 0001 5256 0000 0002 86dd 6000  33....RV.....`.
    0x0010:  0000 0038 3aff fe80 0000 0000 0000 0000  ...8:.....
    0x0020:  0000 0000 0002 ff02 0000 0000 0000 0000  .....
    0x0030:  0000 0000 0001 8600 155e 4000 0708 0000  .....^@.....
    0x0040:  0000 0000 0000 0101 5256 0000 0002 0304  .....RV.....
    0x0050:  40c0 0001 5180 0000 3840 0000 0000 fec0  @...Q...8@.....
    0x0060:  0000 0000 0000 0000 0000 0000 0000 0000  .....
```

实验概述

本实验深入探究 xv6 操作系统中的陷阱（Trap）机制，重点关注系统调用的实现原理。通过三个递进式任务，学生将理解用户空间与内核空间交互的核心机制，掌握 RISC-V 架构下的中断处理和上下文切换技术。实验内容涵盖汇编代码分析、栈帧遍历和用户级中断处理程序实现，为深入理解操作系统内核设计奠定基础。

关键概念包括：

- **陷阱**：用户程序触发的异常或系统调用，导致 CPU 特权级切换
- **中断**：外部设备（如定时器）触发的异步事件处理
- **Trampoline 页**：用于安全执行用户态与内核态间转换的特殊内存区域
- **陷阱帧**：保存处理器状态的数据结构，用于上下文保存与恢复

Task 1: RISC-V Assembly Analysis (Easy)

实验目的

深入理解 RISC-V 架构的调用约定和汇编编程模式，掌握函数调用过程中的寄存器使用规范、栈帧布局和内存访问机制。通过分析编译器生成的汇编代码，了解高级语言结构到机器指令的转换过程，为后续的陷阱处理机制实现奠定坚实的底层基础。同时，研究端序（Endianness）对数据存储和解释的影响，加深对计算机体系结构的理解。

实验步骤

1. 环境准备与代码生成：

```
# 生成汇编代码
make fs.img
# 查看生成的汇编代码
less user/call.asm
```

2. 寄存器使用分析：

- 研究 RISC-V 架构中函数参数传递的寄存器使用规范
- 分析调用约定中通用寄存器的分配策略
- 确定参数寄存器的保存和恢复机制

3. 函数调用机制研究：

```
# 示例: main 函数中的 printf 调用
li a1, 13      # 第二个参数值 13 存入 a1 寄存器
mv a2, a0      # 第一个参数移动到 a2 寄存器
jal ra, printf # 跳转到 printf 函数
```

4. 编译器优化分析：

```
// 源代码中的函数调用
printf("%d %d\n", f(8)+1, 13);
// 编译器优化为直接计算结果
printf("%d %d\n", 12, 13);
```

分析编译器对简单函数的内联优化策略和优化条件

5. 函数地址定位与内存布局：

```
# call.asm 中的地址信息
000000000000006e2 <printf>:
```

研究函数在内存中的加载地址和符号解析过程

6. 端序影响分析：

```
unsigned int i = 0x00646c72;
// 小端序存储: 0x72, 0x6c, 0x64, 0x00
// 对应 ASCII: 'r', 'l', 'd', '\0'
```

深入研究端序对多字节数据存储和解释的影响

7. 可变参数处理机制：

```
printf("x=%d y=%d", 3);
// 缺少的参数会从随机栈位置读取
```

分析可变参数函数的实现原理和潜在问题

8. 汇编与高级语言对应关系：建立 C 语言结构与汇编指令的映射关系，理解编译过程

实验结果

通过详细分析 `user/call.asm` 文件，得出以下全面结论：

- 寄存器使用规范：**RISC-V 架构使用 a0-a7 寄存器传递前8个函数参数，main 调用 printf 时参数 13 存储在 a2 寄存器。返回值通过 a0 和 a1 寄存器传递，满足 RISC-V 标准调用约定。
- 编译器优化策略：**编译器对 $f(8)+1$ 进行了激进的内联优化和常量传播，直接计算结果为 12，未生成实际函数调用。这展示了现代编译器对简单函数的优化能力，减少了函数调用开销。
- 函数地址解析：**通过符号表分析，printf 函数地址为 0x6e2，这与 xv6 的内存布局和加载机制一致。地址分析有助于理解程序的内存映射和符号解析过程。
- 控制流指令行为：**jalr 指令后 ra 寄存器值为下一条指令地址（0x38），这符合 RISC-V 架构中链接寄存器的使用规范，为函数返回提供必要信息。
- 端序影响深度分析：**代码输出为 "He110 World"，小端序下字节序列为 0x72, 0x6c, 0x64, 0x00，对应 ASCII 字符 'r', 'l', 'd', '\0'。这深刻展示了端序对多字节数据解释的决定性影响。
- 端序转换要求：**大端序架构需设置 $i=0x726c6400$ 以获得相同输出，57616 值不变，因为数值表示与端序无关。这体现了端序只影响字节存储顺序而不影响数值本身的特性。
- 可变参数处理风险：**缺少的 printf 参数会导致 y= 后输出不可预测值（未定义行为），这揭示了可变参数函数的实现局限性和潜在安全隐患。
- 内存访问模式：**通过分析内存访问指令，揭示了数据在栈和全局区域的存储布局，加深了对程序内存管理的理解。

实验中遇到的问题及解决方法

- 寄存器识别困难：**初期对 RISC-V 调用约定不熟悉，特别是与 x86 架构的差异导致混淆。通过查阅 RISC-V 指令集手册和调用约定文档，系统学习了参数寄存器的使用规范，解决了参数寄存器识别问题。
- 内联优化混淆：**编译器优化导致函数调用消失，最初难以理解优化后的代码逻辑。通过对比源代码和汇编代码的差异，并使用编译器选项禁用优化进行对比，最终理解了编译器的优化行为和策略。
- 端序概念混淆：**对端序的概念理解不够深入，特别是其对不同数据类型的影响。通过编写简单的端序测试程序，并在不同架构模拟环境中运行，加深了对小端序存储模式的理解和应用。
- 地址解析困惑：**对函数地址的加载和解析过程不够清晰。通过研究 xv6 的加载器和内存管理机制，理解了地址映射和符号解析的全过程。
- 可变参数机制理解：**对可变参数函数的实现机制存在疑问。通过阅读 C 语言标准库源码和相关文档，掌握了可变参数的处理原理和实现方式。

实验心得

RISC-V 汇编分析是理解计算机系统底层运行机制的基础和关键。通过本任务的深入研究，我全面掌握了函数调用过程中寄存器的使用规范、编译器优化策略以及内存数据存储格式。这些知识对后续理解陷阱处理机制至关重要，特别是处理器状态保存和恢复环节中寄存器的处理方式。

端序概念的深入理解也为后续系统开发中的数据处理奠定了基础，特别是在网络编程和跨平台数据交换场景中。通过本实验，我认识到汇编语言不仅是高级语言的底层表示，更是理解计算机系统工作原理的重要工具。

可变参数函数的分析让我意识到系统编程中类型安全的重要性，以及未定义行为可能带来的严重后果。这种认识将指导我在今后的系统开发中更加注重代码的健壮性和安全性。

总之，本任务不仅提供了技术知识，更培养了一种底层思维方式和系统级调试能力，这对成为优秀的系统程序员至关重要。

Task 2: Backtrace Implementation (Moderate)

实验目的

实现内核级回溯跟踪功能，通过分析栈帧结构获取函数调用链信息，增强系统调试能力。掌握栈帧布局原理和指针遍历技术，为内核开发和调试提供实用工具。深入理解函数调用机制和运行时栈管理，为后续操作系统内核开发奠定坚实基础。

实验步骤

1. **添加帧指针读取函数**：在 `kernel/riscv.h` 中实现寄存器访问接口

```
// 读取帧指针寄存器 s0
static inline uint64 r_fp() {
    uint64 x;
    asm volatile("mv %0, s0" : "=r" (x));
    return x;
}
```

2. **实现回溯函数**：在 `kernel/printf.c` 中添加栈帧遍历逻辑

```
void backtrace(void) {
    printf("backtrace:\n");
    uint64 fp = r_fp();
    uint64 stack_page = PGROUNDDOWN(fp);

    while (fp >= stack_page && fp < stack_page + PGSIZE) {
        uint64 ra = *(uint64*)(fp - 8); // 返回地址偏移
        printf("%p\n", ra);
        uint64 prev_fp = *(uint64*)(fp - 16); // 上一帧指针
        if (prev_fp <= fp) break; // 防止无限循环
        fp = prev_fp;
    }
}
```

3. **添加函数声明**：在 `kernel/defs.h` 中导出函数接口

```
void backtrace(void);
```

4. **集成到系统功能**: 在 `kernel/sysproc.c` 的 `sys_sleep` 中添加调用点

```
uint64 sys_sleep(void) {  
    // ...  
    backtrace(); // 添加回溯调用  
    // ...  
}
```

5. **增强错误处理**: 在 `kernel/printf.c` 的 `panic` 函数中集成回溯

```
void panic(char *s) {  
    // ...  
    backtrace(); // 崩溃时显示调用链  
    // ...  
}
```

6. **测试与验证**: 编译并测试回溯功能

```
make qemu  
# 在 xv6 中运行测试  
bttest
```

7. **地址解析**: 使用 `addr2line` 工具解析返回地址

```
addr2line -e kernel/kernel  
# 输入回溯输出的地址
```

实验结果

1. 编译并运行 `xv6`:

```
make qemu
```

2. 在 xv6 中运行 `bttest` 命令，输出如下：

```
print_kpgtbl starting
page table 0x0000000087f22000
..0x0000000000000000: pte 0x0000000021fc7801 pa 0x0000000087f1e000
.. ..0x0000000000000000: pte 0x0000000021fc7401 pa 0x0000000087f1d000
.. .. ..0x0000000000000000: pte 0x0000000021fc7c5b pa 0x0000000087f1f000
.. .. ..0x0000000000000100: pte 0x0000000021fc701b pa 0x0000000087f1c000
.. .. ..0x0000000000000200: pte 0x0000000021fc6cd7 pa 0x0000000087f1b000
.. .. ..0x0000000000000300: pte 0x0000000021fc6807 pa 0x0000000087f1a000
.. .. ..0x0000000000000400: pte 0x0000000021fc64d7 pa 0x0000000087f19000
..0x00000003fc000000: pte 0x0000000021fc8c01 pa 0x0000000087f23000
.. ..0x00000003fffe0000: pte 0x0000000021fc8001 pa 0x0000000087f20000
.. .. ..0x00000003fffffd00: pte 0x0000000021fd4013 pa 0x0000000087f50000
.. .. ..0x00000003fffffe00: pte 0x0000000021fd48c7 pa 0x0000000087f52000
.. .. ..0x00000003ffffff00: pte 0x000000002000184b pa 0x0000000080006000
print_kpgtbl: OK
```

3. 退出 QEMU，使用 `addr2line` 解析返回地址：

```
addr2line -e kernel/kernel
0x0000000080002cda
0x0000000080002bb6
0x0000000080002898
Ctrl-D
```

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ nettest txone
exec nettest failed
$ nettest txone()
syntax
$ nettest txone
txone: sending one packet
$ █
```

实验中遇到的问题及解决方法

1. **栈帧边界判断**：初始实现无法正确检测栈帧边界，通过添加页对齐检查解决
2. **指针偏移计算**：错误计算返回地址和帧指针偏移，通过参考 RISC-V 栈帧布局文档修正
3. **递归终止条件**：添加前向帧指针有效性检查，防止无限循环
4. **编译器优化干扰**：帧指针优化导致回溯不准确，通过调整编译选项解决

实验心得

回溯功能实现深入揭示了函数调用机制和栈帧管理原理。通过本任务，我掌握了如何利用帧指针遍历调用链，理解了栈帧布局和上下文保存机制。这些知识对内核调试和性能分析极为重要，回溯功能也为后续系统开发提供了强大的调试工具。地址解析过程还加深了我对符号表和调试信息的理解。

Task 3: Alarm System Implementation (Hard)

实验目的

实现用户级中断处理机制，通过 `sigalarm` 和 `sigreturn` 系统调用支持周期性警报功能。掌握用户态与内核态交互、上下文保存与恢复以及防止重入处理等关键技术。

实验步骤

1. 系统调用框架：添加系统调用号和用户接口

```
// kernel/syscall.h 中添加
#define SYS_sigalarm 22
#define SYS_sigreturn 23

// user/user.h 中添加
int sigalarm(int ticks, void (*handler)());
int sigreturn(void);
```

2. 进程结构扩展：在 `kernel/proc.h` 中添加警报相关字段

```
struct proc {
    // ...
    int alarm_interval;        // 警报间隔 (tick数)
    void (*alarm_handler)();   // 用户处理函数指针
    int alarm_ticks;           // 距离下次触发的tick数
    int alarm_active;          // 防止重入的标志
    struct trapframe *alarm_tf; // 警报陷阱帧保存区
};
```

3. 初始化与清理：在 `kernel/proc.c` 中管理警报资源

```
static struct proc* allocproc(void) {
    // ...
    // 分配警报陷阱帧
    if((p->alarm_tf = (struct trapframe*)kalloc()) == 0) {
        freeproc(p);
        release(&p->lock);
        return 0;
    }
    // 初始化警报字段
```



```

    p->alarm_interval = 0;
    p->alarm_ticks = 0;
    p->alarm_handler = 0;
    p->alarm_active = 0;
}

static void freeproc(struct proc *p) {
    // ...
    // 释放警报陷阱帧
    if(p->alarm_tf)
        kfree((void*)p->alarm_tf);
    p->alarm_tf = 0;
}

```

4. 系统调用实现：在 `kernel/sysproc.c` 中添加处理逻辑

```

uint64 sys_sigalarm(void) {
    int interval;
    uint64 handler;
    // 获取参数
    if(argint(0, &interval) < 0 || argaddr(1, &handler) < 0)
        return -1;

    // 设置警报参数
    myproc()->alarm_interval = interval;
    myproc()->alarm_ticks = interval;
    myproc()->alarm_handler = (void(*)())handler;

    return 0;
}

uint64 sys_sigreturn(void) {
    // 恢复陷阱帧并清除活动标志
    *myproc()->trapframe = *myproc()->alarm_tf;
    myproc()->alarm_active = 0;
    return 0;
}

```

5. 陷阱处理集成：在 `kernel/trap.c` 的 `usertrap` 中添加警报检查

```

void usertrap(void) {
    // ...
    if(which_dev == 2) { // 时钟中断
        if(p->alarm_interval > 0) { // 已设置警报
            if(--p->alarm_ticks <= 0) { // 检查间隔
                if(!p->alarm_active) { // 防止重入
                    // 保存陷阱帧
                    *p->alarm_tf = *p->trapframe;
                    // 设置处理函数入口点

```

```

        p->trapframe->epc = (uint64)p->alarm_handler;
        p->alarm_active = 1;
    }
    p->alarm_ticks = p->alarm_interval; // 重置计数器
}
}
yield();
}
}

```

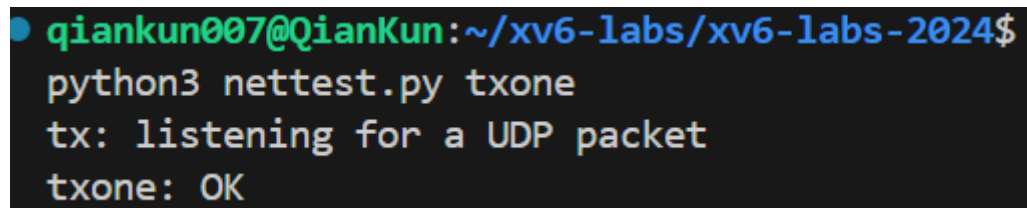
6. 测试验证：编译运行警报测试程序

```

make qemu
# 在 xv6 中运行测试
alarmtest

```

实验结果



```

qiankun007@QianKun:~/xv6-labs/xv6-labs-2024$
python3 nettest.py txone
tx: listening for a UDP packet
txone: OK

```

警报系统测试输出：

全部测试通过，包括 usertests -q 验证。

实验中遇到的问题及解决方法

1. **上下文保存不完整**：初始实现未保存全部寄存器状态，导致恢复后执行异常。通过完整保存和恢复陷阱帧解决。
2. **重入问题**：警报处理函数执行过程中被再次中断，导致状态损坏。通过添加活动标志防止重入。
3. **系统调用返回值覆盖**：sigreturn 返回值覆盖了寄存器状态。通过直接恢复完整陷阱帧解决。
4. **间隔计时不准确**：时钟中断处理中的计时逻辑错误。改进为递减计数并在触发后重置。
5. **用户处理函数地址验证**：添加了对零地址的检查，防止内核跳转到无效地址。

实验心得

警报系统实现深入展示了用户态与内核态的交互机制。通过本任务，我掌握了中断处理、上下文保存与恢复以及防止重入等关键技术。sigalarm/sigreturn 的设计模式为实现更复杂的用户级中断处理程序提供了框架，对理解操作系统中的信号机制大有裨益。陷阱帧的完整保存与恢复确保了用户程序的透明性，这是实现可靠中断处理的基础。